

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Periodic Real-Time Scheduling Simulator: RM, DM, and EDF

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Embedded Systems

Contents

1 Part 1: theoretical investigation 2

2 Processor calculations 2

3 Part 2: scheduling implementation 7

1 Part 1: theoretical investigation

Answer each question with a reasoned discussion. Compare single-processor, multicore, and system-on-chip embedded architectures, with benefits, limitations, and application examples. Compare a real-time operating system with general-purpose Linux/Windows in scheduling, memory management, interrupt handling, and determinism. Explain priority inversion and compare priority inheritance with priority-ceiling protocols. Distinguish hard and soft real-time requirements and give industrial/commercial examples. Discuss reliability and fault tolerance in industrial and automotive embedded systems: watchdogs, redundancy, checkpointing, and error-detection codes, including both advantages and limitations. Discuss memory, computation, and energy constraints in TinyML/edge AI and explain quantization, pruning, and model compression.

2 Processor calculations

Processor A runs at 2 GHz and executes a 12-million-instruction program. Processor B runs at 1.5 GHz and executes a nine-million-instruction program. Use the four category weights and CPI values below. Calculate weighted CPI for both cases, execution time on each processor, and performance in MIPS. State the interpretation of the provided percentages as instruction-mix weights.

Category	**Weight (percent)**	**CPI, case 1**	**CPI, case 2**
1	40	2	3
2	30	3	2
3	10	4	3
4	20	5	4

Table 2: Processor calculations - numerical data and parameters.

Task	**Assigned frequency (%)**	**CPI: case 1**	**CPI: case 2**
1	40	2	3
2	30	3	2
3	10	4	3
4	20	5	4

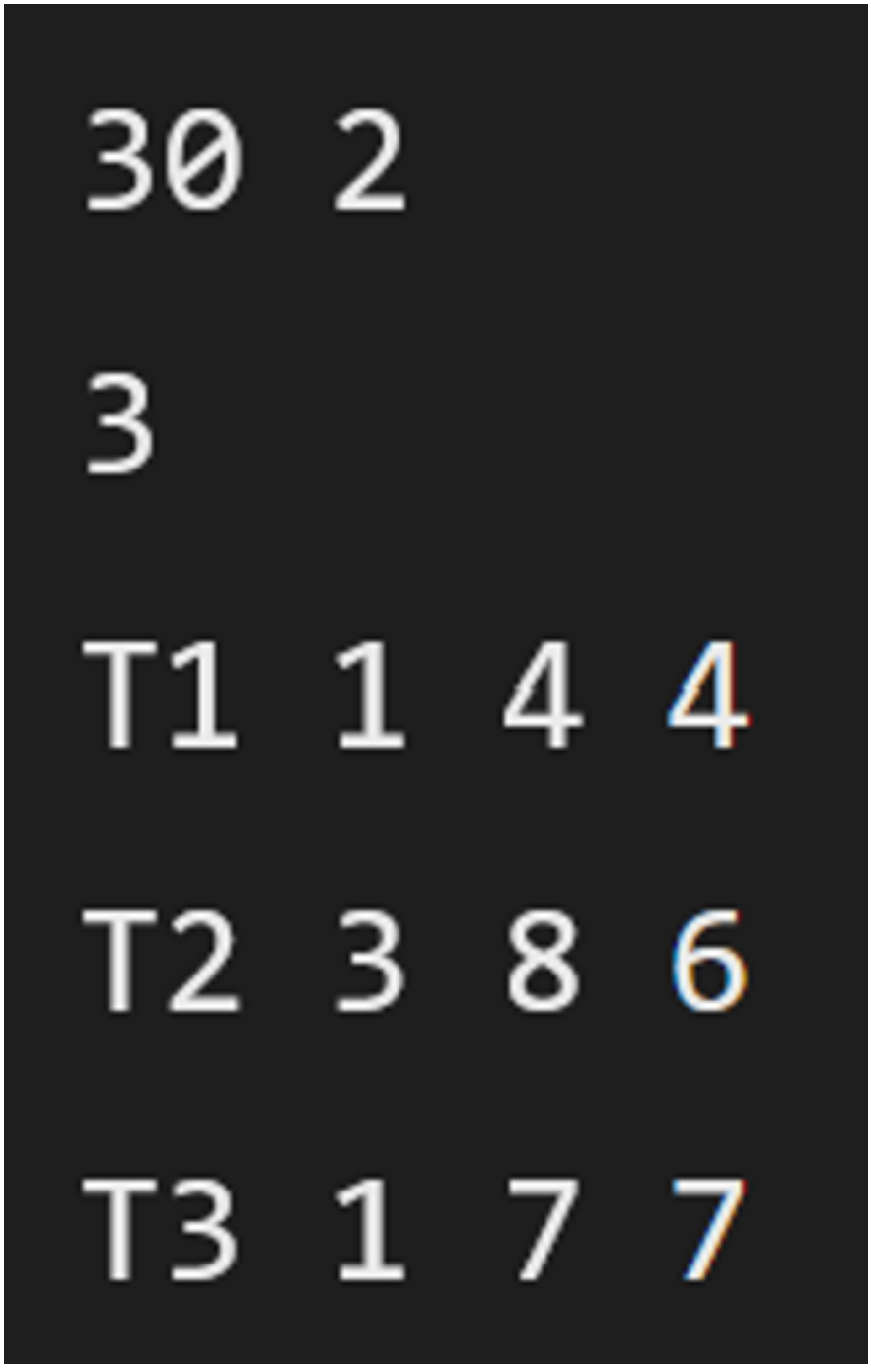


Figure 1: Diagram or recorded result: Processor calculations.

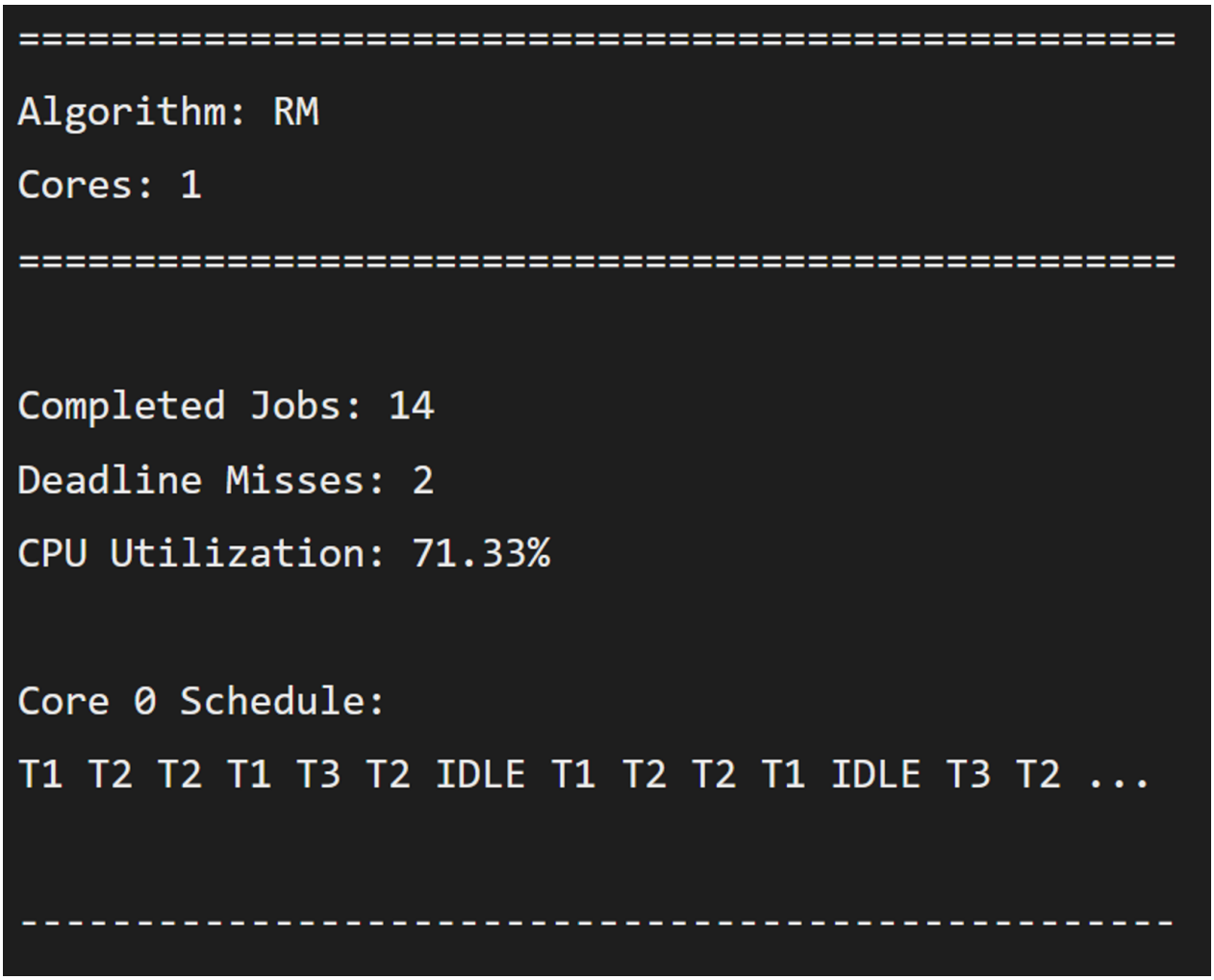


Figure 2: Diagram or recorded result: Processor calculations.

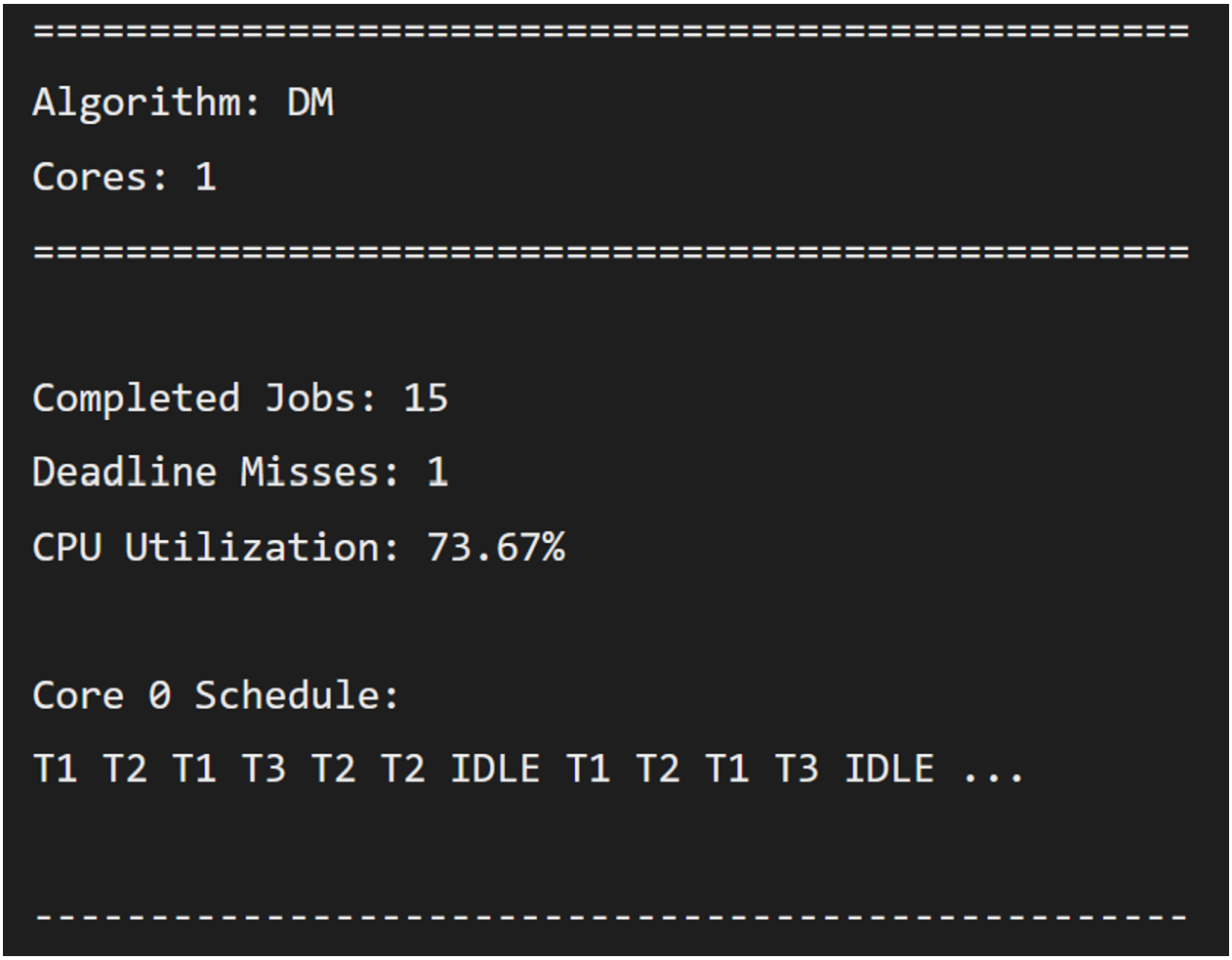


Figure 3: Diagram or recorded result: Processor calculations.

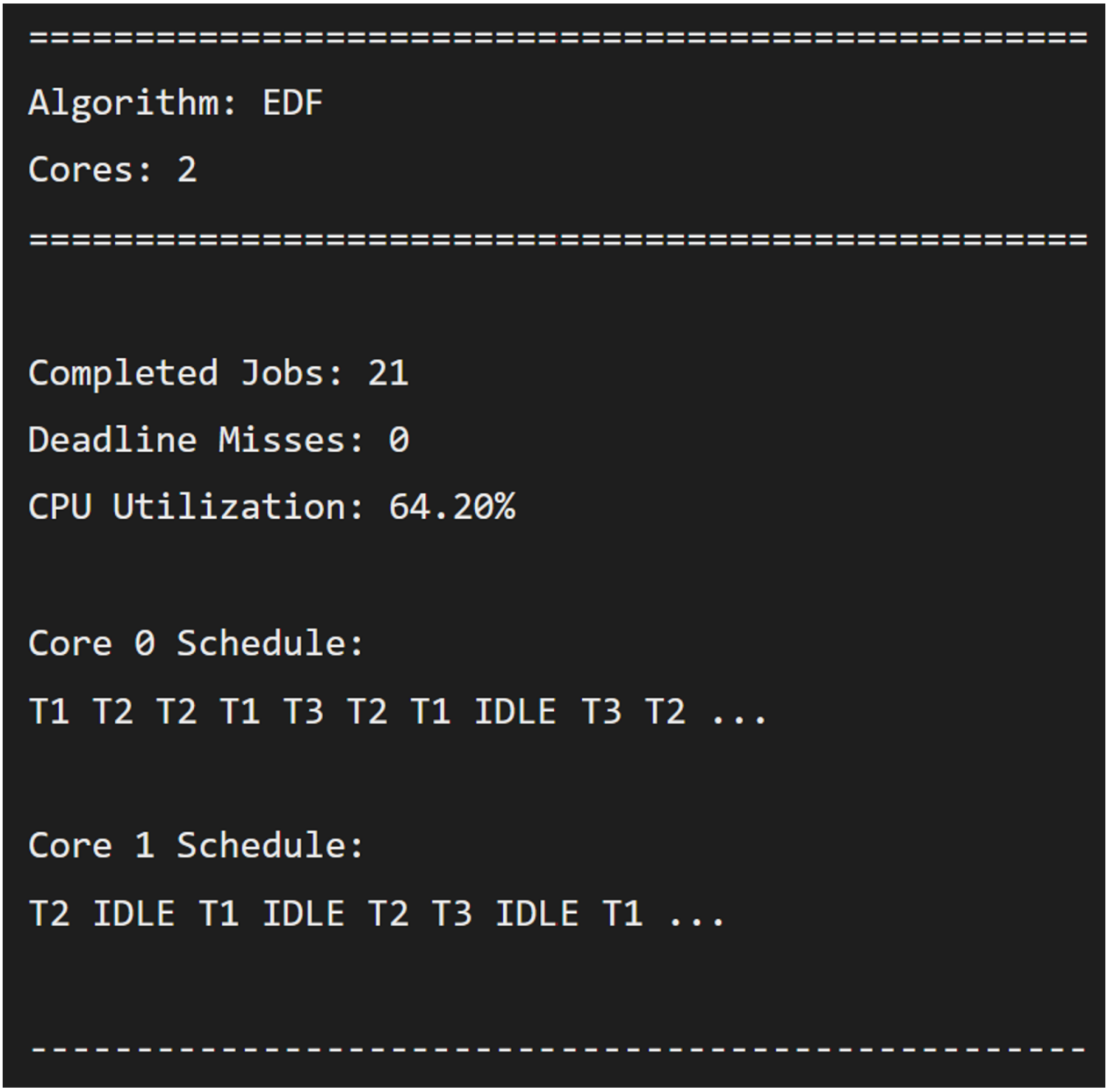


Figure 4: Diagram or recorded result: Processor calculations.

3 Part 2: scheduling implementation

Implement a stepwise simulation of periodic tasks, each described by execution time C, period T, and relative deadline D. Implement Rate Monotonic, Deadline Monotonic, and Earliest Deadline First for one core. Extend the simulator to two cores with a shared ready queue, permitting simultaneous execution of distinct jobs. Compare completed jobs, deadline misses, and utilization. Read input.txt: the first line specifies simulation duration and core count; the second gives the number of tasks; later lines give name, execution time, period, and relative deadline. Write a text output for each algorithm containing algorithm name, core count, completed-job count, misses, utilization, and each core's time schedule. Produce a separate Gantt chart for each algorithm/core configuration, with consistent colors for each task. Analyze differences in utilization and missed deadlines, the circumstances under which EDF can behave better, and the effect of extra cores. Submit Python source, input, text output, report, and plots.

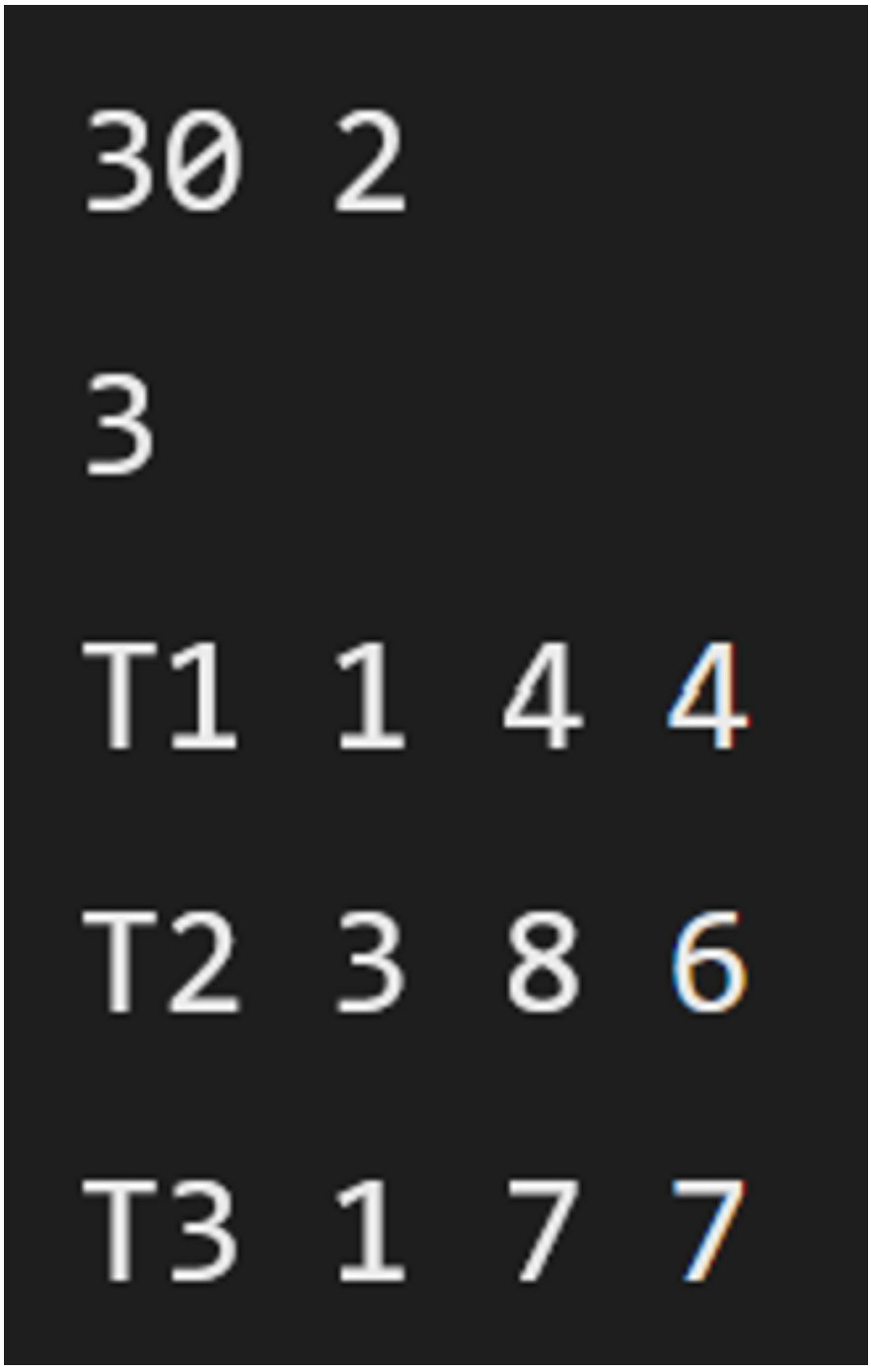


Figure 5: Diagram or recorded result: Part 2: scheduling implementation.

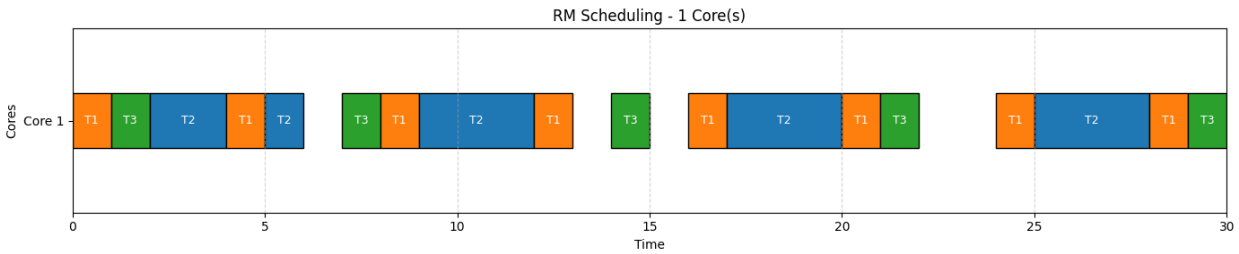


Figure 6: Diagram or recorded result: Part 2: scheduling implementation.

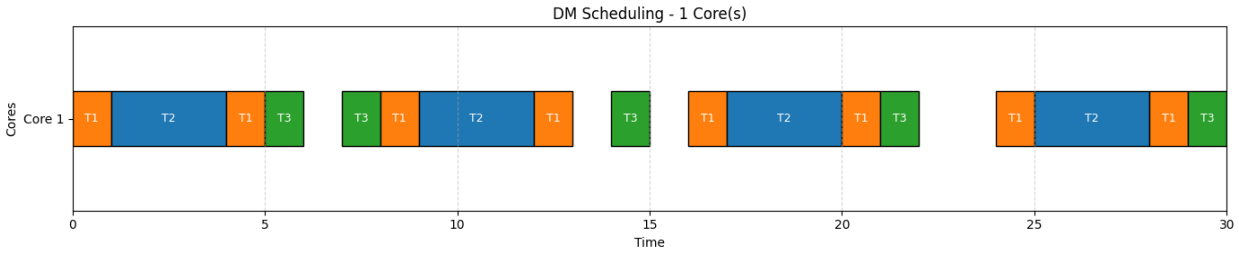


Figure 7: Diagram or recorded result: Part 2: scheduling implementation.

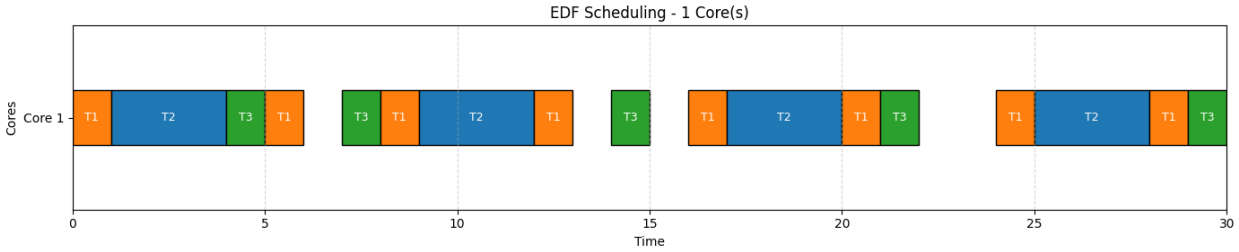


Figure 8: Diagram or recorded result: Part 2: scheduling implementation.

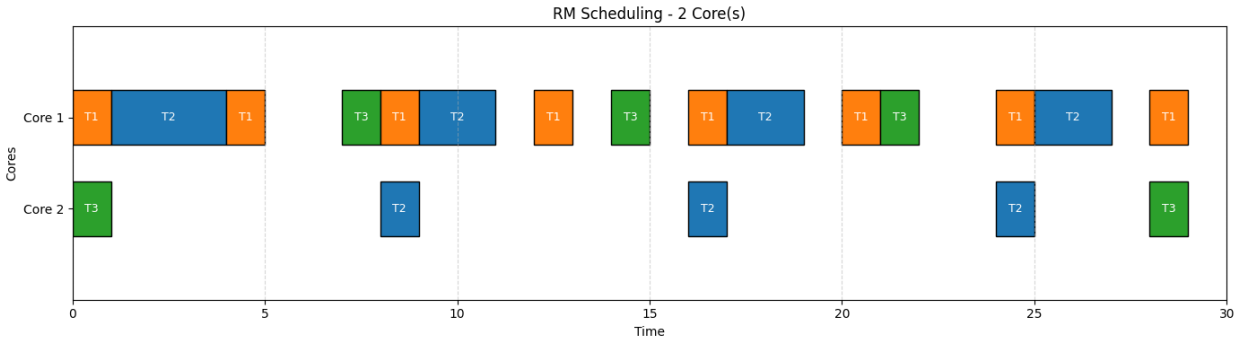


Figure 9: Diagram or recorded result: Part 2: scheduling implementation.

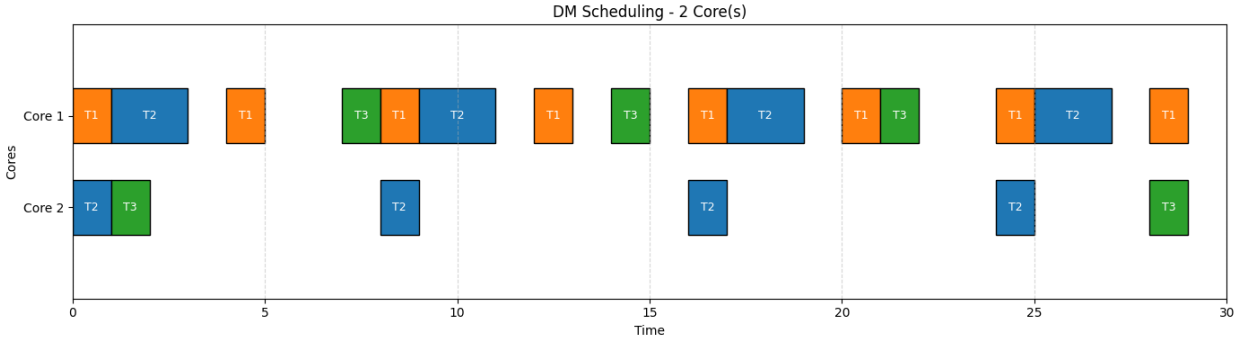


Figure 10: Diagram or recorded result: Part 2: scheduling implementation.

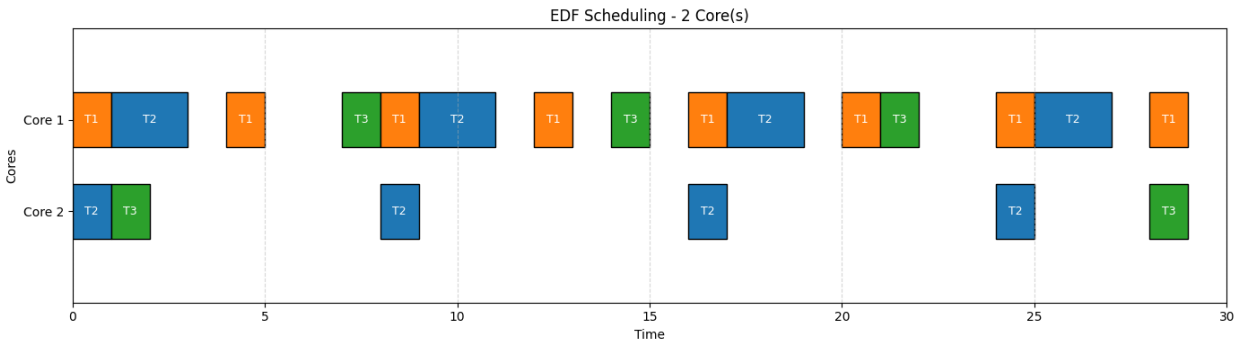


Figure 11: Diagram or recorded result: Part 2: scheduling implementation.